

End Report: COMP.SE.140 Project – Fall 2024

Instructions for the Teaching Assistant

The project implements a continuous deployment pipeline for a microservices-based application, extending the previous nginx exercise. This report details the implementation, testing procedures, and deployment specifications.

Implemented Optional Features

- Testing of individual services
 - in project's root /tests/ directory
- Static analysis step in the pipeline
 - using pylint, created another step/stage in the pipeline which checks static analysis.
- Monitoring and logging for troubleshooting.
 - not implemented.
- Deployment to external server.
 - The project implements deployment to an external server, specifically the CSC virtual machine. This deployment option uses SSH keys for secure access and includes automated deployment through the GitLab CI/CD pipeline. The deployment is accessible through the public IP address <http://195.148.22.168:8198/> , allowing testing via browser and curl commands without requiring SSH login.

Instructions for Testing the System

The application can be accessed through two primary interfaces:

Web Interface: <http://195.148.22.168:8198/>

API Gateway: <http://195.148.22.168:8197/>

The RESTful API provides the following endpoints:

1. State Management:

```
...  
# Get current state  
curl http://195.148.22.168:8197/state  
    —or—  
curl localhost:8197/state  
  
# Change state (INIT/RUNNING/PAUSED/SHUTDOWN)  
curl localhost:8197/state -X PUT -d "PAUSED" -H "Content-Type: text/plain" -H "Accept:  
text/plain"  
    —or—  
curl http://195.148.22.168:8197/state -X PUT -d "PAUSED" -H "Content-Type: text/plain" -H  
"Accept: text/plain"  
...
```

2. Run Log:

```
...  
# View state transition history  
curl http://195.148.22.168:8197/run-log  
    —or—  
curl localhost:8197/run-log  
...
```

3. Request Test:

```
...  
# Test service functionality  
curl http://195.148.22.168:8198/request  
...
```

Development Environment

Development Platform:

- Hardware: MacBook Pro M1 (Apple Silicon)
- Architecture: ARM64
- Operating System: macOS Sonoma 14.6.1 (23G93)
- Docker Version: 24.0.6
- Docker Compose Version: 2.21.0
- GitLab Runner Version: 17.8.1

Description of the CI/CD Pipeline

- Version Management

The project uses Git for version control with a dedicated 'project' branch for development. This separation ensures that the previous nginx exercise remains accessible for evaluation while new features are implemented. All development work is conducted through GitLab, with the repository maintaining a clear commit history that demonstrates the test-driven development approach.

- Building Tools

The build process utilizes Docker and Docker Compose for containerization. The application consists of multiple services:

1. Nginx Gateway: Alpine-based Nginx container serving as the API gateway
2. Service1: Python-based service with three load-balanced instances
3. Service2: Node.js-based service
4. Load Balancing: Implemented through Nginx configuration

- Testing Framework

Testing is implemented using pytest with the following structure:

1. Service Tests:
 - test_service1.py: Tests for the main service functionality
 - test_service2.py: Tests for the secondary service
 - test_api_state.py: Tests for state management API
2. Test Categories:
 - Unit tests for individual service functions
 - Integration tests for API endpoints
 - State transition tests

- Packing

The application is packaged using Docker containers with specific configurations for each service:

1. Nginx Container:
 - Base: nginx:alpine
 - Custom configuration for API gateway
 - Load balancing configuration
2. Service Containers:

- Python-based containers for Service1
- Node.js container for Service2
- Shared network configuration

- Deployment

Deployment is automated through GitLab CI/CD pipeline with four stages:

1. Linting Stage:
 - checks service1.py and service2.js for lint errors.
 - only runs/continues the pipeline if there are no errors or if lint score is 8/10 in service1.py.
2. Build Stage:
 - Constructs Docker images
 - Verifies build integrity
3. Test Stage:
 - Runs automated tests
 - Verifies service functionality
4. Deploy Stage:
 - Copies configuration to CSC VM
 - Builds and starts containers
 - Verifies deployment

Example Pipeline Runs

- Successful Pipeline Example:

...

\$ gitlab-ci job output

Stage 1/3: build ✓

[+] Building docker images

[+] Docker Compose build complete

Stage 2/3: test ✓

[+] Running pytest

[+] All tests passed

Stage 3/3: deploy ✓

[+] Deployment to CSC VM successful

Pipeline completed successfully

...

- Failed Pipeline Example:

...

```
$ gitlab-ci job output
Stage 1/3: build ✓
Stage 2/3: test ✗
Error: test_state_transitions failed
AssertionError: Expected state 'RUNNING', got 'PAUSED'
Pipeline failed
```
```

## Reflections

### - Main Learnings

The project provided valuable insights into several key areas:

#### 1. CI/CD Pipeline Development:

Understanding the practical aspects of continuous deployment, from automated testing to deployment automation.

#### 2. Container Orchestration:

Managing multiple containers and their interactions, particularly in a load-balanced environment.

#### 3. Cloud Deployment:

Working with cloud infrastructure and understanding the security implications of public-facing services.

### - Challenges Faced

The most significant challenges included:

#### 1. State Management Implementation:

Ensuring consistent state across multiple service instances required careful consideration of race conditions and synchronization.

#### 2. Cloud Platform Configuration:

Setting up proper security groups and networking on CSC ePouta required understanding of cloud infrastructure concepts.

#### 3. Test Framework Integration:

Developing comprehensive tests that could run reliably in the CI/CD pipeline while covering all critical functionality.

## Improvement Suggestions

Given more time, several improvements could be implemented:

1. Enhanced Monitoring:  
Adding proper logging and monitoring tools for better operational visibility.
2. Automated Rollback:  
Implementing automatic rollback mechanisms for failed deployments.
3. Security Enhancements:  
Implementing proper secrets management and additional security measures.

## Effort Estimation

Total time spent on the project: approximately 40-50 hours

### **Distribution:**

- Initial Setup and Planning: 8 hours
- Implementation: 20 hours
- Testing and Debugging: 12 hours
- CI/CD Pipeline Setup: 6 hours
- Documentation: 2-3 hours